Programación Concurrente y Distribuida

Tema 2. Programas concurrentes

Infografía Tema 2

Programación Concurrente: Del Caos del Indeterminismo al Control Sincronizado

El Desafío: Indeterminismo y Falta de Atomicidad

Las sentencias de alto nivel no son atómicas

Una operación simple como `n = n + 1` se descompone en múltiples instrucciones de ensamblador.

El orden parcial altera el resultado final

Diferentes secuencias de ejecución de los mismos procesos pueden producir valores de memoria inconsistentes.

Recursos No Compartibles

Elementos (como una variable o impresora) que requieren acceso exclusivo para ser modificados.

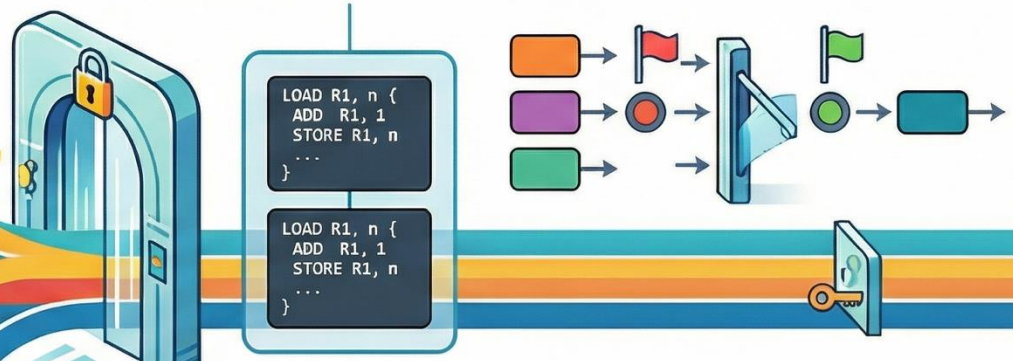
La Solución: Secciones Críticas y Sincronización

La Sección Crítica (SC)

Código que accede a recursos no compartibles y debe ejecutarse de forma atómica.

Protocolos de Entrada y Salida

Verifican que el recurso esté libre antes de entrar y notifican su liberación al saiir.

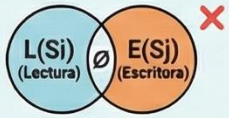


Seguridad y Vivacidad

Garantizar que nada maio pase (Exclusión Mutus)

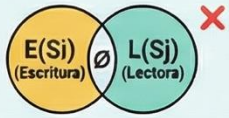
y que algo bueno ocurra eventualmente.

Condiciones de Bernstein (Diseño Seguro de Concurrencia)



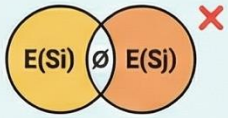
$L(Si) \cap E(Sj) = \emptyset$

Lo que lee el proceso i no debe ser modificado por el proceso j.



$E(Si) \cap L(Sj) = \emptyset$

Lo que escribe el proceso i no debe ser leído por el proceso j.



$E(Si) \cap E(Sj) = \emptyset$

Ambos procesos no deben escribir en el mismo recurso.

NotebookLM

Programa concurrente

- Consiste en la ejecución simultánea de dos o más procesos/hilos
- Los procesos/hilos colaboran para realizar una tarea global

Integrales secuencial

```
Calcular Integral de f(x);
Calcular Integral de g(x);
```

Integrales concurrente

Proceso 1

```
Calcular Integral de f(x);
```

Proceso 2

```
Calcular Integral de g(x);
```

Programa concurrente

Contar palabras secuencial

```
int contador = 0;
Fichero f = siguienteFichero();
while (f!= null) {
  int palabras = contarPalabras(f);
  contador = contador + palabras;
  f = siguienteFichero();
}
```

Contar palabras concurrente

Proceso 1

```
Fichero f = siguienteFicheroNoMarcado();
while (f != null) {
  marcaFichero(f);
  int palabras = contarPalabras(f);
  contador = contador + palabras;
  f = siguienteFicheroNoMarcado();
}
```

Proceso 2

```
Fichero f = siguienteFicheroNoMarcado();
while (f != null) {
  marcaFichero(f);
  int palabras = contarPalabras(f);
  contador = contador + palabras;
  f = siguienteFicheroNoMarcado();
}
```

¡INCORRECTO!

Memoria global

```
contador = 0;
```

Indeterminismo: Orden total vs Orden Parcial

Programa 1

```
1) ParaCaprichos= 0,5 * Saldo;
2) Saldo = Saldo - ParaCaprichos;
```

Memoria global

```
Saldo = 100;
```

Programa 2

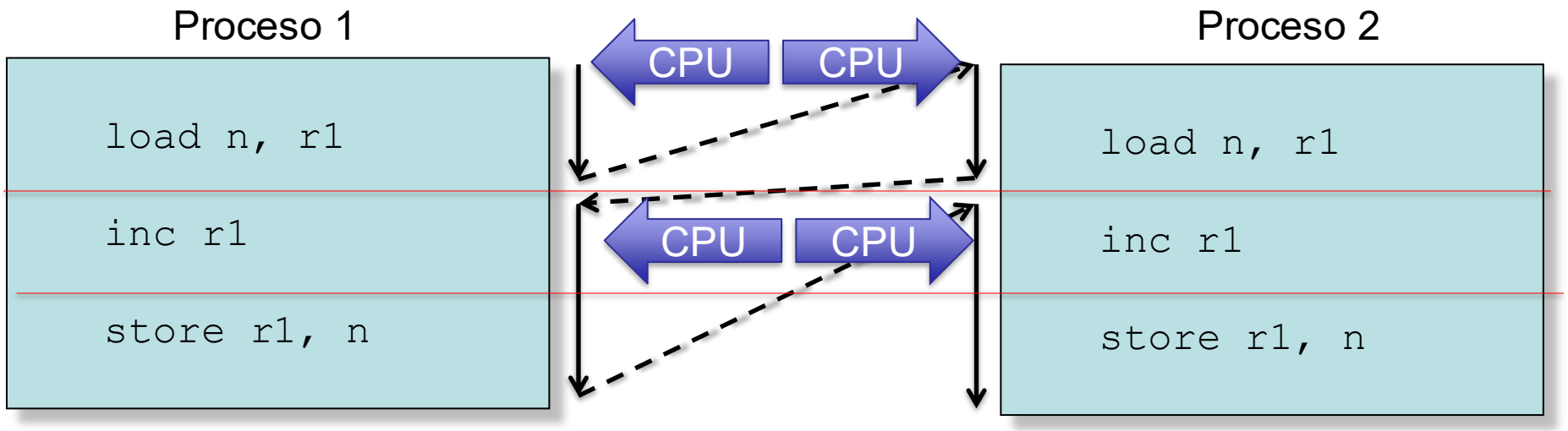
```
a) Ahorro = 20;
b) Saldo = Saldo + Ahorro
c) ParaCaprichos = 0,1 * Saldo;
d) Saldo = Saldo - ParaCaprichos;
```

- Posibles secuencias sin concurrencia:
  - 1, 2, a, b, c, d
  - a, b, c, d, 1, 2
- Posibles secuencias concurrentes:
  - 1, a, b, c, 2, d
  - a, b, c, 1, 2, d
  - …
- ¿Cuál es el Saldo?
  - 1, 2, a, b, c, d = 63
  - a, b, c, d, 1, 2 = 54
  - 1, a, b, c, 2, d = 58
  - a, b, c, 1, 2, d = 48

No Atomicidad e Indeterminismo

- La cosa se puede complicar aún más
- Las sentencias en lenguaje de alto nivel **no son atómicas**

- También podemos encontrar el problema del orden parcial a nivel ensamblador (Interleaving)



Ejercicio

- Si n es una variable global inicializada a 0 y ejecuto dos procesos con el siguiente código:

```
for ( int i = 0 ; i < 10 ; i++ )
            n = n + 1;
```

¿cuál será el valor final de n?

- Si el valor no es siempre el mismo ¿cuál es el menor y el mayor posible?

Recursos no compartibles

Hardware

- Pantalla
- Impresora
- Disco duro
- Memoria

Software

- Variable
- Permiso

Recursos no compartibles son los que requieren de un acceso en modo exclusivo para realizar operaciones de modificación o escritura

Conceptos Importantes:

- **Sección Crítica**
  - Exclusión basada en acceso a un recurso (la SC son todos los trozos de código donde se accede a ese recurso)
    - **Exclusión Mutua** (**solo un proceso**)
      - En muchos casos entendemos por EM que solo puede estar UN proceso
      - P.e. Acceso a la pantalla.
    - Exclusión Mutua que permite varios procesos
      - Hay situaciones en que un recurso es compartible por algunos (no todos) procesos.
      - Normalmente en función del tipo de proceso o el número de procesos que ya hay en la sección crítica.
      - P.e. Acceso de lectura a un disco que no está siendo escrito.
- **Condiciones de sincronización**
  - Es una exclusión basada en que se dé una determinada condición (trozos de código que solo se pueden ejecutar si se cumple algo)
    - P.e. Acceder a un buffer solo si hay elementos en dicho buffer.
- **Corrección en los programas**
  - Los programas deben ser seguros y vivaces.

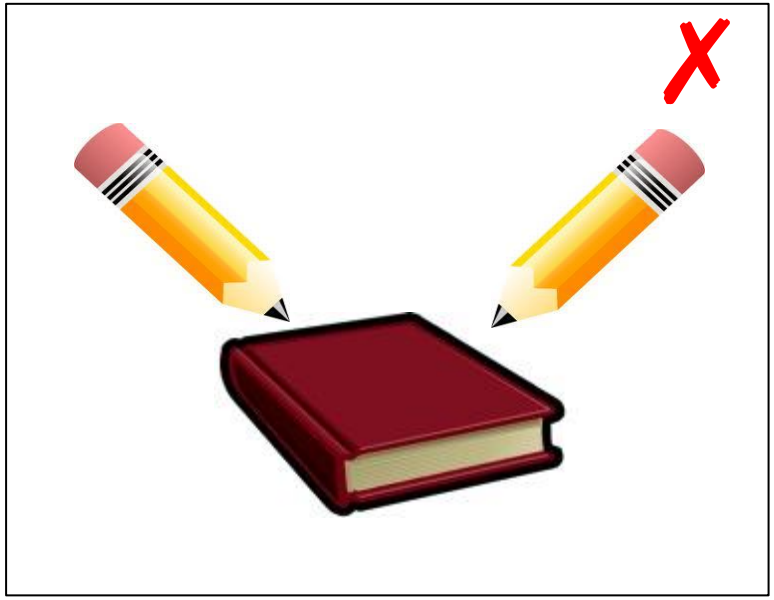
Sección Crítica

- Conjunto de instrucciones que acceden en modo lectura y/o escritura a los mismos recursos NO compartibles.
- Las Condiciones de Bernstein permiten determinar qué parte del código se puede ejecutar de forma concurrente y cuál no.
- Dos conjuntos de instrucciones puede ejecutarse concurrentemente siempre que los recursos NO compartibles en los que escriben sean distintos.
- Las secciones críticas están asociadas a UN RECURSO x NO COMPARTIBLE.
- Una sección crítica es un conjunto de instrucciones que queremos que se ejecuten de manera atómica (en exclusión mutua)

Condiciones de Bernstein

- $L(S_k)=\{a_1,a_2,\ldots,a_n\}$ es el **cjto de lectura** del cjto de instrucciones S_k, y son todas las variables a_i que se leen durante la ejecución de S_k.
- $E(S_k)=\{b_1,b_2,\ldots,b_n\}$ es el cjto **de escritura**.
- Las condiciones que deben cumplirse son:

$$L(S_i) \cap E(S_j) = \emptyset$$
$$E(S_i) \cap L(S_j) = \emptyset$$
$$E(S_i) \cap E(S_j) = \emptyset$$



Ejercicio

Memoria global

```
int n = 0;
int p = 5;
int r = 0;
int s = 2;
```

Proceso 1

```
println(n);
int i = 0;
while ( i < n ) {
  r = r + s
  i = i + 1;
}
```

Proceso 2

```
n = 5;
int i = 2;
println(i);
if (r > 3 || s < 2)
  println(n);
p = p - s;
```

- Identifica las distintas secciones críticas y los problemas que pueden aparecer debidos al indeterminismo
- ¡No se te olvide la pantalla!

Protección de secciones críticas

- Es necesario garantizar que sólo un proceso pueda estar ejecutando la Sección Crítica (SCx) de un recurso x.
- Antes de ejecutar la SCx se debe estar seguro de que el recurso está libre.
- Después de ejecutar la SCx se debe informar al resto de procesos interesados que el recurso vuelve a estar libre.
- Cada recurso(s) tendrá(n) sus SCs particulares.
- Al tipo de **sincronización** donde **solo un proceso** puede estar ejecutándose a la vez le llamamos **exclusión mutua**.

Proceso 1

```
// Protocolo Entrada SCx
SECCIÓN CRÍTICA RECURSO x
// Protocolo Salida SCx
```

...

Proceso N

```
// Protocolo Entrada SCx
SECCIÓN CRÍTICA RECURSO x
// Protocolo Salida SCx
```

Protocolos entrada/salida de sección crítica

Deben cumplir estas condiciones

- **Exclusión mutua**
  - Ejecución exclusiva de la sección crítica.
- **Limitación en la espera**
  - Ningún proceso debe esperar de forma indefinida para entrar en SC.
- **Progreso en la ejecución**
  - Si la SC está libre un proceso que quiera entrar debe poder hacerlo.

Exclusión mutua

- Garantizar que **sólo un proceso** accede al mismo tiempo a un recurso no compartible (R).
- Mientras un proceso está en su sección crítica relativa a R ningún otro proceso podrá estar ejecutando secciones críticas relativas a R.
- Mientras **R está siendo utilizado** por un proceso, **los demás procesos** que deseen ejecutar una sección crítica relativa a R deberán quedar **bloqueados** a la espera de que el primero termine.
- Si R está libre, cualquier proceso (y solo 1) podrá entrar a su sección crítica relativa a R.
- Los protocolos de entrada/salida deben garantizar limitación en la espera y progreso en la ejecución.

- Exclusión mutua

- **Condición de sincronización**

- Corrección de programas

Condición de sincronización

- Garantizar la ejecución en exclusión mutua de las secciones críticas puede no ser suficiente

Proceso 1

```
n = n + 1;
```

Memoria Global

```
n = 0;
```

Proceso 2

```
int y = n - 1;
```

- ¿Qué pasa si un objeto de datos compartidos está en un estado inapropiado para ejecutar una acción determinada? (p.e. imagina que **n** nunca pueda ser negativa).
- El proceso 2 debe esperar a que n tenga un valor apropiado por las acciones de otros procesos.
- Aquí tenemos otro concepto importante en concurrencia:

  Una **condición de sincronización**

Condición de sincronización

Las condiciones de sincronización son las relaciones de precedencia existentes entre los distintos procesos de una aplicación concurrente que deben cumplirse para que el resultado sea coherente

Captar Imagen

Guardar en Buffer

Buffer

Leer del Buffer

Procesar y almacenar como JPG

- Exclusión mutua
- Condición de sincronización
- **Corrección de programas**

Corrección de los programas

- Seguridad (Nada malo va a pasar)
  - Garantizar Exclusión Mutua
  - Garantizar Sincronización
  - Evitar Interbloqueos pasivos (deadlocks)
- Vivacidad (Algo bueno pasará, eventualmente)
  - Evitar Interbloqueos activos (livelock)
  - Evitar Inanición

Ejercicio 1

Memoria global

```
int x = 100;
```

Proceso 1

```
x = x + 10;
```

Proceso 2

```
if (x > 100) {
  println(x);
} else {
  println(x-50);
}
```

- En el siguiente programa se espera que como resultado de su ejecución se imprima 110 ó 50. ¿es correcta la suposición?

WOOCLAP

Parte 1

Implementación de Exclusión Mutua (y Condición de Sincronización)

- Inhibición de interrupciones
- Basadas en memoria compartida
  - Espera ocupada
  - Semáforos
  - Monitores
- Basadas en paso de mensajes
  - Paso de mensajes send/receive
  - Llamadas a procedimientos remotos
  - Invocaciones remotas

Implementación de Exclusión Mutua

Requisitos de un protocolo de entrada (cualquiera):

- **Limitación en la espera**
- **Progreso en la ejecución**
- Válida para múltiples procesos
  - Que funcione para número indeterminado de procesos
- Equitativa (si no hay prioridades en el problema)
  - No haya preferencias de unos procesos sobre otros

EXCLUSIÓN MUTUA (P_i procesos que contienen una <u>sección crítica sobre el mismo recurso</u> no compartible)

```
// Protocolo Entrada SCx
SECCIÓN CRÍTICA de P1 para Rx
// Protocolo Salida SCx
```

...

```
// Protocolo Entrada SC
SECCIÓN CRÍTICA de Pn para Rx
// Protocolo Salida SC
```

Implementación de Exclusión Mutua

(y Condición de Sincronización)

Implementación de EXCLUSIÓN MÚTUA mediante software:

- Se tienen P_i procesos que contienen una sección crítica sobre **el mismo recurso** no compartible Rx.
- **Protocolos de entrada/salida**: Secuencia de sentencias que enmarcan la sección crítica
  - Protocolo de Entrada: Comprueba condición que autoriza la entrada a SCx.
  - Protocolo de Salida: Indica que se ha terminado de ejecutar la SCx.

P_1

```
// Protocolo Entrada SCx
SECCIÓN CRÍTICA de P1 para Rx
// Protocolo Salida SCx
```

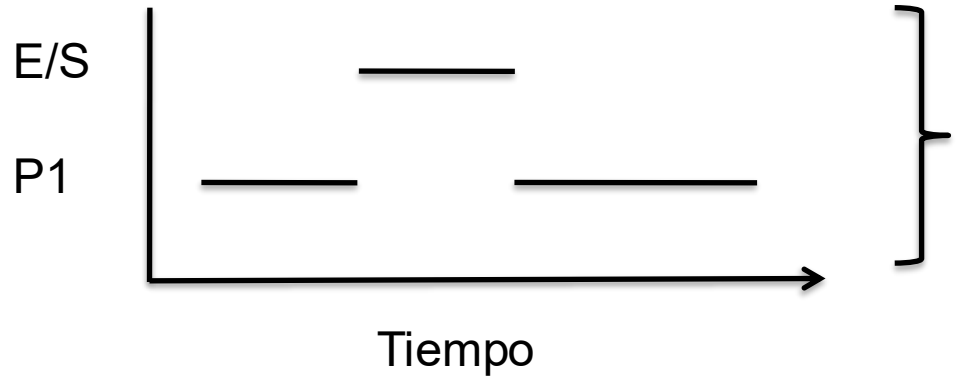
...

P_n

```
// Protocolo Entrada SCx
SECCIÓN CRÍTICA de Pn para Rx
// Protocolo Salida SCx
```

Inhibición de interrupciones

(Monoprocesadores)



Ejecución “simultánea” de dos procesos gracias a las interrupciones

- Los cambios de contexto también pueden implementarse así
- Solución:

```
Deshabilitar Interrupciones;
Sección crítica;
Habilitar Interrupciones;
```

- Sólo es válido en sistemas monoprocesador

Espera ocupada

- Consiste en **mantener ocupada la CPU** mientras que **no se cumpla** la condición necesaria para continuar.
- La exclusión mutua se implementa teniendo como condición el hecho de que no haya ningún proceso en la sección crítica.
- ¿Cómo?:
  - Pues comprobando continuamente el valor de una o más variables compartidas que indiquen este hecho.
- La evaluación de la condición sirve para determinar si hay que seguir esperando o no.

```
while (!condicion)      // <- Espera ocupada | Protocolo
          {};                  //             |    de
Condicion = false;                   //        |  Entrada
// Sección crítica o acción que esperaba condición
condicion = true;                //   | Protocolo de salida
// Resto proceso
```

Primer intento de exclusión mutua

Proceso 1

```
while (true) {
 // Protocolo de entrada
  while (ocupada)
          {};    // do nothing
  ocupada = true;
  // Aquí Sección crítica
 // Protocolo de salida
  ocupada = false;
  // Resto proceso
}
```

Proceso 2

```
while (true) {
 // Protocolo de entrada
  while (ocupada)
          {};    // do nothing
  ocupada = true;
  // Aquí Sección crítica
 // Protocolo de salida
  ocupada = false;
  // Resto proceso
}
```

- La variable **ocupada** indica el estado de la sección crítica, es global, compartida e inicialmente vale **false**.
- Fijaos que podría haber expresado la condición en “positivo” (y usar `libre` y el while sería `while(!libre)` y luego `libre=false` en protocolo de entrada, y `libre= true` en protocolo de salida.
- Este intento es incorrecto porque no proporciona exclusión mutua,

Segundo intento de exclusión mutua

Proceso 1

```
while (true) {
 // Protocolo de entrada
  while (turno!= 1)
          {};
  // Sección crítica
// Protocolo de salida
  turno = 2;
  // Resto proceso
}
```

Proceso 2

```
while (true) {
  // Protocolo de entrada
   while (turno != 2)
          {};
  // Sección crítica
// Protocolo de salida
  turno = 1;
  // Resto proceso
}
```

- La variable **turno** indica el proceso que tiene derecho a acceder al recurso no compartible, es global, compartida y su valor inicial puede ser cualquiera: 1 o 2.
- Este intento es incorrecto porque no permite progreso en la ejecución

Cuarto intento de exclusión mutua

Proceso 1

```
while (true) {
 // Protocolo de entrada
  quiereEntrar1= true;
  while (quiereEntrar2)
          {};
  // Sección crítica
// Protocolo de salida
  quiereEntrar1 = false;
  // Resto proceso
}
```

Proceso 2

```
while (true) {
 // Protocolo de entrada
  quiereEntrar2 = true;
  while (quiereEntrar1)
          {};
  // Sección crítica
// Protocolo de salida
  quiereEntrar2 = false;
  // Resto proceso
}
```

- Las variables quiereEntrarX indican el interés del proceso X por acceder al recurso o que ya lo tiene (**true**), o el hecho de que ya no lo necesitan (**false**). Son globales, compartidas e inicialmente valen **false**.
- Este intento es incorrecto porque puede provocar espera infinita

Algoritmo correcto de Peterson

Proceso 1

```
while (true) {
 // Protocolo de entrada
  quiereEntrar1 = true;
  turno = 2;
  while ( quiereEntrar2
      && (turno == 2))
           {};
  // Sección crítica
 // Protocolo de salida
  quiereEntrar1 = false;
  // Resto proceso
}
```

Proceso 2

```
while (true) {
 // Protocolo de entrada
  quiereEntrar2 = true;
  turno = 1;
  while ( quiereEntrar1
      && (turno == 1))
           {};
  // Sección crítica
 // Protocolo de salida
  quiereEntrar2 = false;
  // Resto proceso
}
```

- Las variables quiereEntrarX indican el interés del proceso X por acceder al recurso o que ya lo tiene (**true**), o el hecho de que ya no lo necesitan (**false**). Son globales, compartidas e inicialmente valen **false**.
- La variable **turno** indica el proceso que tiene derecho a acceder al recurso no compartible, es global, compartida y su valor inicial puede ser cualquiera: 1 o 2

Corrección de los programas

- Seguridad
  - Garantizar Exclusión Mutua
  - Garantizar Sincronización
  - **Evitar Interbloqueos pasivos (deadlocks)**
- Vivacidad
  - **Evitar Interbloqueos activos (livelock)**
  - **Evitar Inanición**

Interbloqueos Pasivos: Deadlock

Proceso 1

```
BloquearSiPantallaOcupada();
ocupar(Pantalla);
BloquearSiImpresoraOcupada();
ocupar(Impresora);
// Muestro e imprimo
liberar(Impresora);
liberar(Pantalla);
```

Proceso 2

```
BloquearSiImpresoraOcupada();
ocupar(Impresora);
BloquearSiPantallaOcupada();
ocupar(Pantalla);
// Muestro e imprimo
liberar(Impresora);
liberar(Pantalla);
```

Todos los procesos están detenidos esperando que ocurra algo que no pasará

Interbloqueos Activos: Livelock

Proceso 1

```
while (!libre(Impresora)) {
  sleep(10);
}
ocupar(Impresora);
// Imprimir
liberar(Impresora);
```

Proceso 2

```
while (true) {
  if (libre(Impresora)) {
    ocupar(Impresora);
    // Imprimir (3 segundos)
    liberar(Impresora);
  }
}
```

Un proceso no está detenido, ejecuta código, pero no consigue progresar en su ejecución (abajo es livelock infinito).

```
while (true) {
  quiereEntrar1= true;
  while (quiereEntrar2)
          {};
  // Sección crítica
  quiereEntrar1 = false;
  // Resto proceso
}
```

```
while (true) {
  quiereEntrar2 = true;
  while (quiereEntrar1)
          {};
  // Sección crítica
  quiereEntrar2 = false;
  // Resto proceso
}
```

Inanición

- Siendo **n** una variable global compartida inicializada a 0

Proceso 1

```
while (n % 2 == 0 ) // espera
          {};   // si n es par
// Realizar cálculos
```

Proceso 2

```
while (true) {
  n = n + 2;
}
```

Algún proceso no puede llegar a progresar nunca

Ejercicio 2. Solución

Memoria global

```
int c = 1;
```

Proceso 1

```
int x = 0;
while ( x == 0) TAS(x);
// Sección Crítica
c = 1;
// Resto proceso
```

Proceso 2

```
int x = 0;
while ( x == 0) TAS(x);
// Sección Crítica
c = 1;
// Resto proceso
```

Ejercicio 2. ¿Solución?

Memoria global

```
int c = 1;
```

Proceso 1

```
int x = 0;
while ( x == 0) {
  x = c;
  c = 0;
}
// Sección Crítica
c = 1;
// Resto proceso
```

Proceso 2

```
int x = 0;
while ( x == 0) {
  x = c;
  c = 0;
}
// Sección Crítica
c = 1;
// Resto proceso
```

Si sustituimos TAS(x) por sus dos instrucciones ¿sigue siendo una solución válida?

Ejercicio 3

Supongamos que disponemos de un contador de energía eléctrica que genera impulsos cada vez que se consume un Kw de energía y queremos implementar un sistema que cuente el número de impulsos generados en una hora, es decir, que cuente el número de Kw consumidos en una hora. Para ello el sistema estará formado por dos procesos.

- –Un proceso acumulador que lleva la cuenta de los impulsos recibidos.
- –Un proceso escritor que escribe en la impresora cada hora los impulsos recibidos.

En una variable compartida por los dos procesos (n) se lleva la cuenta de los impulsos. Si el sistema se encuentra en un estado correspondiente al valor de la variable n=x y, en estas condiciones, se produce (a la vez) un nuevo impulso y el final de período de una hora, obtener las posibles secuencias de ejecución de los procesos y cuáles de ellas son correctas.

Ejercicio 3. Solución

Memoria global en el instante actual

```
int n = N;
```

Proceso Acumulador

```
while (true) {
  // Espera impulso
  n++;
}
```

Proceso Contador

```
while (true) {
  // Espera 1h
  println(n);
  n = 0;
}
```

1. n++; println(n); n = 0; (CORRECTO)
2. prinlnt(n); n++; n = 0; (INCORRECTO)
3. println(n); n = 0 ; n++; (CORRECTO)
4. ¿Alguna más?
    1. n++, en ensamblador es load, inc y store…

Ejercicio 4

- Supongamos que dos procesos concurrentes P1 y P2 intentan usar el mismo dispositivo de E/S. Concretamente, supongamos que P1 intenta leer del bloque 20 del dispositivo y P2 intenta escribir en el bloque 88.
- Si las operaciones read y write no fueran atómicas y se descomponen en las básicas (saltar y leer o saltar y escribir) se podrían producir problemas.
- En este caso, indicar las distintas secuencias de ejecución posibles para P1 y P2 razonando cuáles son correctas y cuáles no.

Ejercicio 4. Solución

1ª secuencia: saltar 20, leer, saltar 88, escribir (correcta)

2ª secuencia: saltar 20, saltar 88, escribir, leer (incorrecta)

3ª secuencia: saltar 88, saltar 20, escribir, leer (incorrecta)

4ª secuencia: saltar 88, escribir, saltar 20, leer (correcta)

5ª secuencia: saltar 20, saltar 88, leer, escribir (incorrecta)

6ª secuencia: saltar 88, saltar 20, leer, escribir (incorrecta)

Ejercicio 5

Memoria global

```
int n = 0;
boolean parar = false;
```

Proceso 1

```
while (!parar) {
  n = n + 1;
  sleep(1);
}
```

Proceso 2

```
sleep(5);
parar = true;
println(n);
```

- ¿Cuál será el valor mostrado por la pantalla?